

- 1) Index.php - точка входа
- подключаем конфиг
 - создаем роуты
 - инициализируем приложение

```
<?php

// dirname(__DIR__) - вышли в корень проекта
require_once dirname( path: __DIR__ ) . '/config/init.php';
require_once HELPERS . '/functions.php';
require_once CONFIG . '/routes.php';

if (PHP_MAJOR_VERSION < 8) {
    die('Необходима версия PHP >= 8');
}

new \wfm\App(); // инициализируем приложение
```

```
Product.php × routes.php × ProductController.php × View.php × AppController.php × Controller.php ×

<?php

use wfm\Router;

// 1й арг: рег.выражение
// 2й: пара контроллер - action

Router::add( regexp: '^admin/?$', ['controller' => 'Main', 'action' => 'index', 'admin_prefix' => 'admin']);
Router::add( regexp: '^admin/(?P<controller>[a-z-]+)/(?P<action>[a-z-]+)?$', ['admin_prefix' => 'admin']);

Router::add( regexp: '^(?P<lang>[a-z]+)?/?product/(?P<slug>[a-z0-9-]+)/?$',
    ['controller' => 'Product', 'action' => 'view']);
Router::add( regexp: '^(?P<lang>[a-z]+)?/?category/(?P<slug>[a-z0-9-]+)/?$', ['controller' => 'Category', 'action' => 'view']);
Router::add( regexp: '^(?P<lang>[a-z]+)?/?search/?$', ['controller' => 'Search', 'action' => 'index']);
Router::add( regexp: '^(?P<lang>[a-z]+)?/?wishlist/?$', ['controller' => 'Wishlist', 'action' => 'index']);
Router::add( regexp: '^(?P<lang>[a-z]+)?/?page/(?P<slug>[a-z0-9-]+)/?$', ['controller' => 'Page', 'action' => 'view']);
```

```
Router.php × init.php × Product.php × routes.php × ProductController.php ×

10 // путь к папке приложения
11 define("APP", ROOT . '/app');
12 // путь к ядру
13 define("CORE", ROOT . '/vendor/wfm');
14 // путь к функциям помощникам
15 define("HELPERS", ROOT . '/vendor/wfm/helpers');
16 // путь к папке с кешом
17 define("CACHE", ROOT . '/tmp/cache');
18 // путь к логам
19 define("LOGS", ROOT . '/tmp/logs');
20 // путь к папке конфига
21 define("CONFIG", ROOT . '/config');
22 // шаблон сайта по умолчанию - его название
23 define("LAYOUT", 'ishop');
24 // address of our website - наша главная страница
25 define("PATH", 'http://new-ishop.loc');
26 // путь к админке
27 define("ADMIN", 'http://new-ishop.loc/admin');
28 // путь к картинке, если у нас не назначена картинка для товара - картинка по умолчанию
29 define("NO_IMAGE", 'uploads/no_image.jpg');
30
31 // подключим автозагрузчик
32 require_once ROOT . '/vendor/autoload.php';
```

2) Класс APP - инициализирует приложение

`self::$app = Registry::getInstance()` - создаем глобальный контейнер где будем хранить данные. То есть в любом месте приложения мы можем написать `App::$app` и сможем получить данные. Для этого мы написали кастомный класс `Registry`. И сразу методом `getParams` наполняем этот контейнер.

Вызываем роутер и передаем в него текущий url.

```
class App
{
    public static $app; // сюда будет записан наш контейнер

    // gleb
    public function __construct()
    {
        $query = trim(urldecode($_SERVER['QUERY_STRING']), characters: '/'); //текущий url-адресс
        new ErrorHandler();
        session_start();
        // инициализируем приложение - объект класса Registry - наш контейнер
        self::$app = Registry::getInstance();
        // заполняем наш контейнер
        $this->getParams();
        //
        echo $query;
        Router::dispatch($query);
    }

    // подключаем параметры для нашего приложения. Создали их в config/params.php
    // usage gleb
    protected function getParams()
    {
        // получаем массив параметров с config/params.php
        $params = require_once CONFIG . '/params.php';
        if (!empty($params)) {
```

```
            foreach ($params as $k => $v) {
                // все параметры записываем в контейнер
                self::$app->setProperty($k, $v);
            }
        }
    }
}
```

```
index.php x App.php x params.php x Router.php x init.php x
1 <?php
2
3 return [
4     'admin_email' => 'admin@new-ishop.loc',
5     'site_name' => 'E-Shop',
6     'pagination' => 3,
7
8     'smtp_host' => 'smtp.mailtrap.io',
9     'smtp_auth' => true,
10    'smtp_port' => 2525,
11    'smtp_username' => '891424f7fec185',
12    'smtp_password' => '19c6ee864a70e5',
13    'smtp_secure' => null, //ssl
14    'smtp_from_email' => '8eab927eca-e1f3c7@inbox.mailtrap.io',
15
16 ];
```

```
init.php x index.php x TSingleton.php x Registry.php x App.php x params.php x
1 <?php
2
3 // dirname(__DIR__) - вывели в корень проекта
4 require_once dirname(__DIR__) . '/config/init.php';
5
6 if (PHP_MAJOR_VERSION < 8) {
7     die('Необходима версия PHP >= 8');
8 }
9
10 new \wfm\App(); // инициализируем приложение
11 //теперь нам доступен контейнер нашего приложения
12 //echo \wfm\App::$app->getProperty('pagination');
13 //\wfm\App::$app->setProperty('test', 'TEST');
14 var_dump(\wfm\App::$app->getProperties());
15
```

```
array(3) ( ["admin_email"]=> string(19) "admin@new-ishop.loc" ["site_name"]=> string(6) "E-Shop" ["pagination"]=> int(2) )
```

вывело все настройки, которые мы положили в контейнер

3) Router

В App вызвали метод роутера `dispatch` и передали в него текущий url. Класс Router хранит все маршруты. Мы их добавили на 1-м шаге когда в `index.php` вызвали `require_once CONFIG . '/routes.php'`. Мы в этом файле добавили маршруты в класс Router с помощью его метода `Add`.

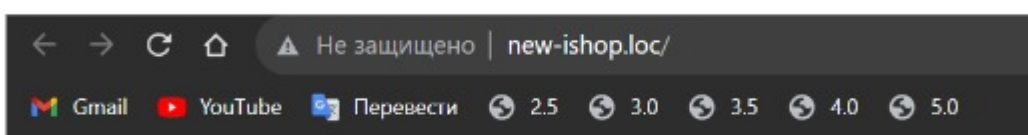
```
class App
{
    public static $app; // сюда будет записан наш контейнер

    /**
     *
     */
    public function __construct()
    {
        $query = trim(urldecode($_SERVER['QUERY_STRING']), characters: '/'); //текущий url
        new ErrorHandler();
        session_start();
        // инициализируем приложение - объект класса Registry - наш контейнер
        self::$app = Registry::getInstance();
        // заполним наш контейнер
        $this->getParams();

        Router::dispatch($query);
    }
}
```

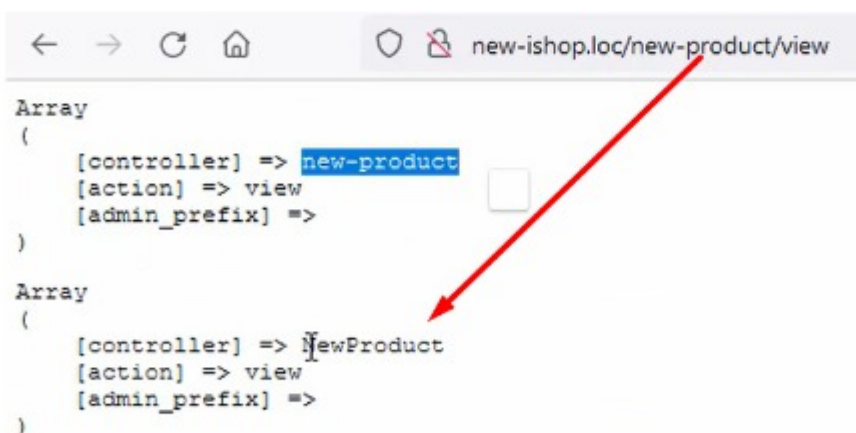
Теперь в массиве `$routes` лежат все эти правила:

```
public static function dispatch($url)
{
    debug( self::$routes, die: 1);
}
```



```
Array
(
    [^admin/?$] => Array
        (
            [controller] => Main
            [action] => index
            [admin_prefix] => admin
        )
    [^admin/(?P[a-z-]+)/(?P[a-z-]+)?$] => Array
        (
            [admin_prefix] => admin
        )
    [^(?P[a-z]+)?/?product/(?P[a-z0-9-]+)?$] => Array
        (
            [controller] => Product
            [action] => view
        )
    [^(?P[a-z]+)?/?category/(?P[a-z0-9-]+)?$] => Array
        (
            [controller] => Category
            [action] => view
        )
    [^(?P[a-z]+)?/?search/?$] => Array
        (
            [controller] => Search
            [action] => index
        )
    [^(?P[a-z]+)?/?wishlist/?$] => Array
        (
            [controller] => Wishlist
            [action] => index
        )
)
```

На выходе с url мы получаем такой `$route` (мы разбили запрашиваемый `$url` и получили с него имена контроллера, екшена, и админ префикс).



Зная имя контроллера создаем его. Для адреса `new-ishop.loc/new-product/view` подключаем контроллер `app\controllers\NewProductController` Также создаем модель для этого контроллера, вызываем action и возвращаем вид.

```
$controller = 'app\controllers\\' . self::$route['admin_prefix'] . self::$route['controller'] . 'Controller';

if (class_exists($controller)) {
    // создаем контроллер и передаем в него текущий путь
    $controllerObject = new $controller(self::$route);

    $controllerObject->getModel();

    $action = self::lowerCamelCase( name: self::$route['action'] . 'Action');
    if (method_exists($controllerObject, $action)) {
        // у контроллера должны вызвать action - метод контроллера
        $controllerObject->$action();
        // подключаем вид
        $controllerObject->getView();
    }
}
```


4) Controller

Мы с контроллера обращаемся к модели и потом полученные данные передаем в вид.

```
class ProductController extends ApplicationController
{
    no usages  gleb*
    public function viewAction()
    {
        $lang = App::$app->getProperty( name: 'language');
        $product = $this->model->get_product($this->route['slug'], $lang);

        if (!$product) {
            // throw new \Exception("Товар по запросу {$this->route['slug']} не найден", 404);
            $this->error_404();
            return;
        }

        $breadcrumbs = Breadcrumbs::getBreadcrumbs($product['category_id'],$product['title']);

        $gallery = $this->model->get_gallery($product['id']);
        $this->setMeta($product['title'], $product['description'], $product['keywords']);
        $this->set(compact( var_name: 'product', ..var_names: 'gallery', 'breadcrumbs'));
    }
}
```

\$lang = App::\$app->getProperty('language') - вот как раз с глобального хранилища (контейнера) получили данные.

Вот тут мы получаем для текущего товара картинки, мета данные и записываем переменные. Но где мы берем переменную \$this->model и методы \$this->setMeta, \$this->set?

Все просто, мы написали абстрактный класс Controller, от которого наследуются все наши контроллеры => поэтому мы в наших контроллерах имеем эти свойства.

Вот абстрактный класс Controller:

```
abstract class Controller
{
    public array $data = []; // массив с данными, которые загружаются из модели и передаются в view
    // чтобы из контроллера в шаблон могли передавать метаданные страницы
    public array $meta = ['title'=>', 'description'=>', 'keywords'=>'];
    1 override
    public false|string $layout = ''; // шаблон
    // с контроллера мы можем переопределить вид. По умолчанию он будет соответствовать названию экрана
    // и будет находиться в папке с названием контроллера
    public string $view = '';
    13 overrides
    public object $model;
```

Мы в нем получаем и храним данные, метаданные, модель, шаблон и вид. У нас есть методы чтобы заполнить эти свойства. И теперь мы можем обращаться в контроллере к его \$this->model.

```
public function getModel()
{
    // путь к модели. модель называется по имени контроллера. имя контроллера находится в route
    $model = 'app\models\\' . $this->route['admin_prefix'] . $this->route['controller'];
    if (class_exists($model)) {
        $this->model = new $model(); // создаем экземпляр модели
        // то есть для контроллера Main есть в папке models соответствующая модель, то мы создадим экземпляр модели
    }
}

no usages  gleb*
public function getView()
{
    // по умолчанию название вида совпадает с названием action
    $this->view = $this->view ?: $this->route['action'];
    // мы получаем данные в Router, потом пробрасываем в контроллер, а из контроллера в view
    // $this->route - текущий маршрут
    // $this->layout - шаблон; $this->view - вид
    (new View($this->route, $this->layout, $this->view, $this->meta))->render($this->data);
}
```

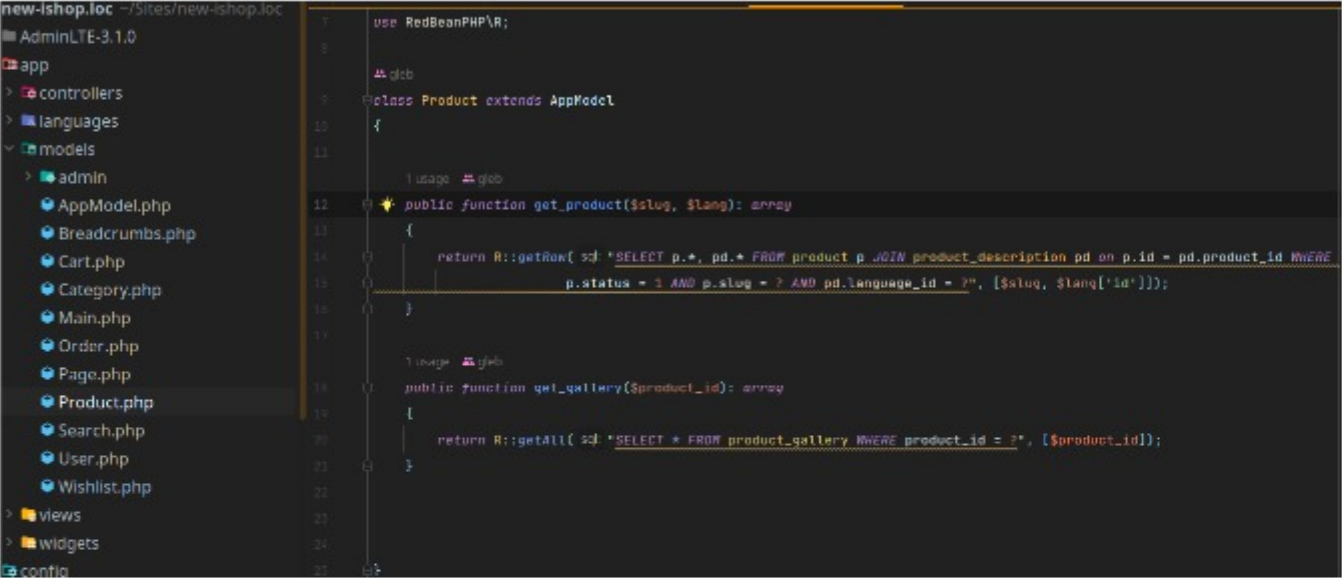
```
gleb
public function set($data)
{
    $this->data = $data; // кладем данные в массив data
}

gleb
public function setMeta($title = '', $description = '', $keywords = '')
{
    $this->meta = [
        'title' => $title,
        'description' => $description,
        'keywords' => $keywords,
    ];
}
```

5) Model

В \$this->model уже лежит экземпляр модели.

Например, в ProductController у нас лежит Product Model.



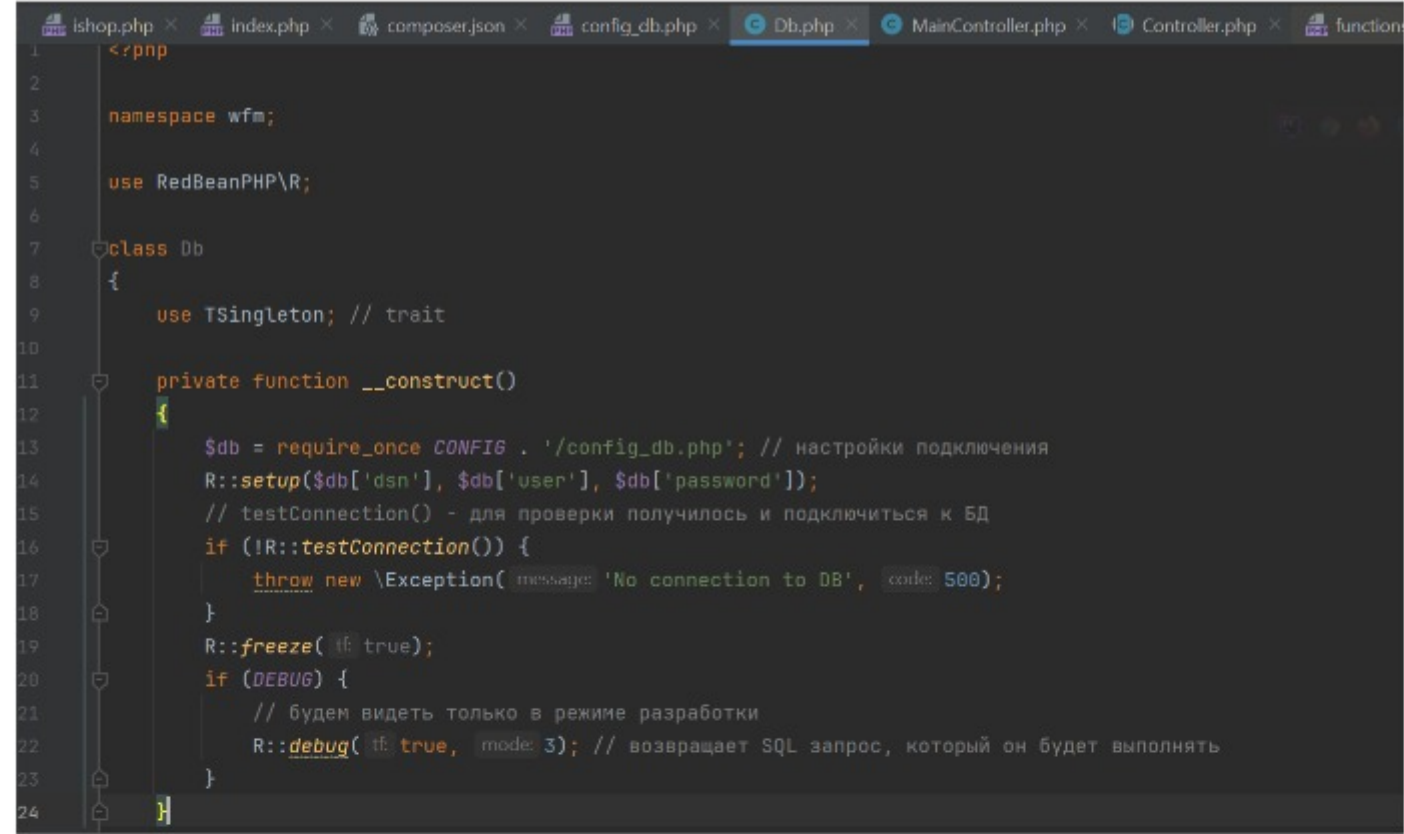
В контроллере получаем все данные с модели и записываем в свойство контроллера - public array \$data = []; // массив с данными, которые загружаются из модели и передаются в view

\$this->set(compact('product', 'gallery', 'breadcrumbs'));

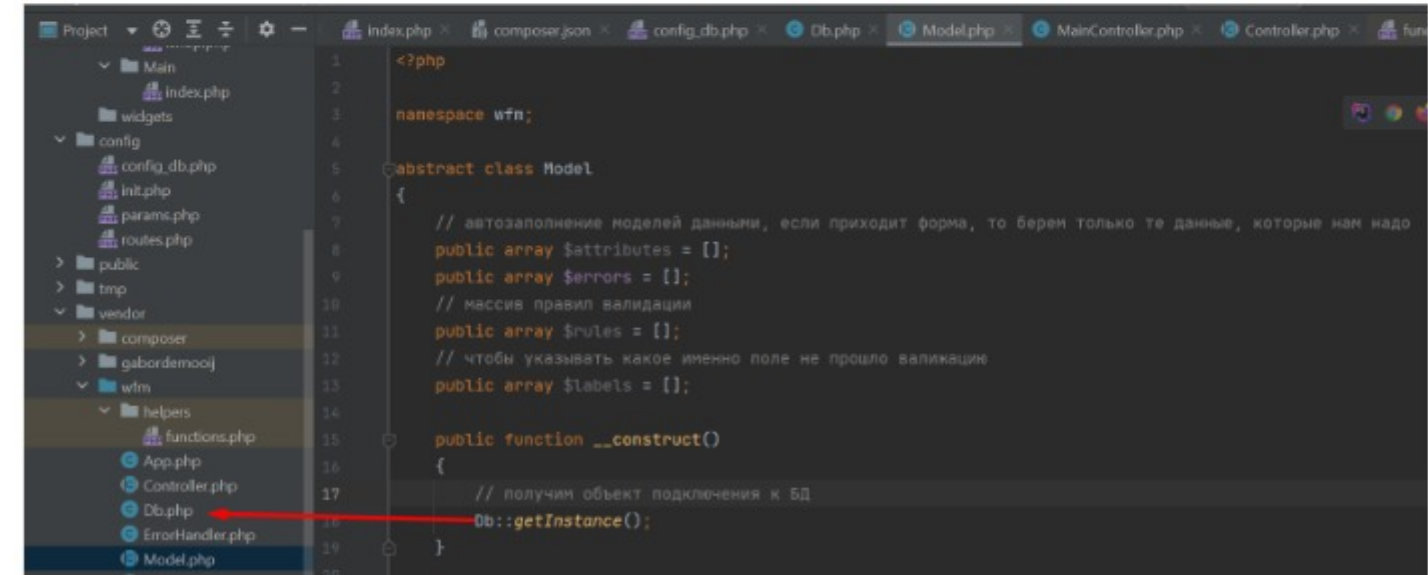
Больше ничего в контроллере не делаем. Данные получены.

Дополнительно:

Мы написали класс, который будет подключаться к БД



Теперь этот класс можем использовать в базовом классе модели.



Для того чтобы в модели было доступно подключение к БД, она должна наследовать базовую модель.

6) View

В контроллере получили все данные. И записали их в переменную \$data.

Дальше в роутере после вызова action контроллера, вызываем View.

```
class Router
{
    public static function dispatch($url)
    {
        if (class_exists($controller)) {
            // создаем контроллер и передаем в него текущий путь
            $controllerObject = new $controller(self::$route);

            $controllerObject->getModel();

            $action = self::lowerCamelCase( name: self::$route['action'] . 'Action');
            if (method_exists($controllerObject, $action)) {
                // у контроллера должны вызвать action - метод контроллера
                $controllerObject->$action();
                // подключаем вид
                $controllerObject->getView();
            }
        }
    }
}
```

У контроллера вызываем метод getView().

```
abstract class Controller
{
    public function getView()
    {
        // по умолчанию название вида совпадает с названием action
        $this->view = $this->view ?: $this->route['action'];
        // мы получаем данные в Router, потом преобразовываем в контроллер, а из контроллера в view
        // $this->route - текущий маршрут
        // $this->layout - шаблон; $this->view - вид
        (new View($this->route, $this->layout, $this->view, $this->meta))->render($this->data);
    }
}
```

Класс View - общий класс, вызываем для всех контроллеров. В нем решаем какой шаблон подключить. Передаем в него \$route, \$layout, \$view, \$meta.

А в метод render(\$this->data) передаем все данные, которые получили в текущей Модели.

Получается метод render подключит нужный вид, нужный шаблон и отрисует страницу.
Контент, который будет отрисовывать попадает в \$content.

Вот мы в контроллере с помощью метода set записали данные.

```
$this->set(compact( var_name: 'product', ..var_names: 'gallery', 'breadcrumbs'));
```

Теперь именно их отрисуем в виде. Например у нас контроллер ProductController, тогда вид app/views/Product/view.php. И в нем отрисуем эти данные.

```
<div class="container">
    <nav aria-label="breadcrumb">
        <ol class="breadcrumb bg-light p-2">
            <li><?php echo $breadcrumbs ?>
        </ol>
    </nav>
</div>
```

```
<div class="col-md-4 order-md-2">
    <h1><?php echo $product['title'] ?></h1>

    <ul class="list-unstyled">
        <li><i class="fas fa-check text-success"></i> В наличии</li>
        <li><i class="fas fa-shipping-fast text-muted"></i> Ожидается</li>
        <li><i class="fas fa-hand-holding-usd"></i> <span class="product-price">
            <?php if ($product['old_price']): ?>
                <small><?php echo $product['old_price'] ?></small>
            <?php endif; ?>
            <?php echo $product['price'] ?>
        </li>
    </ul>

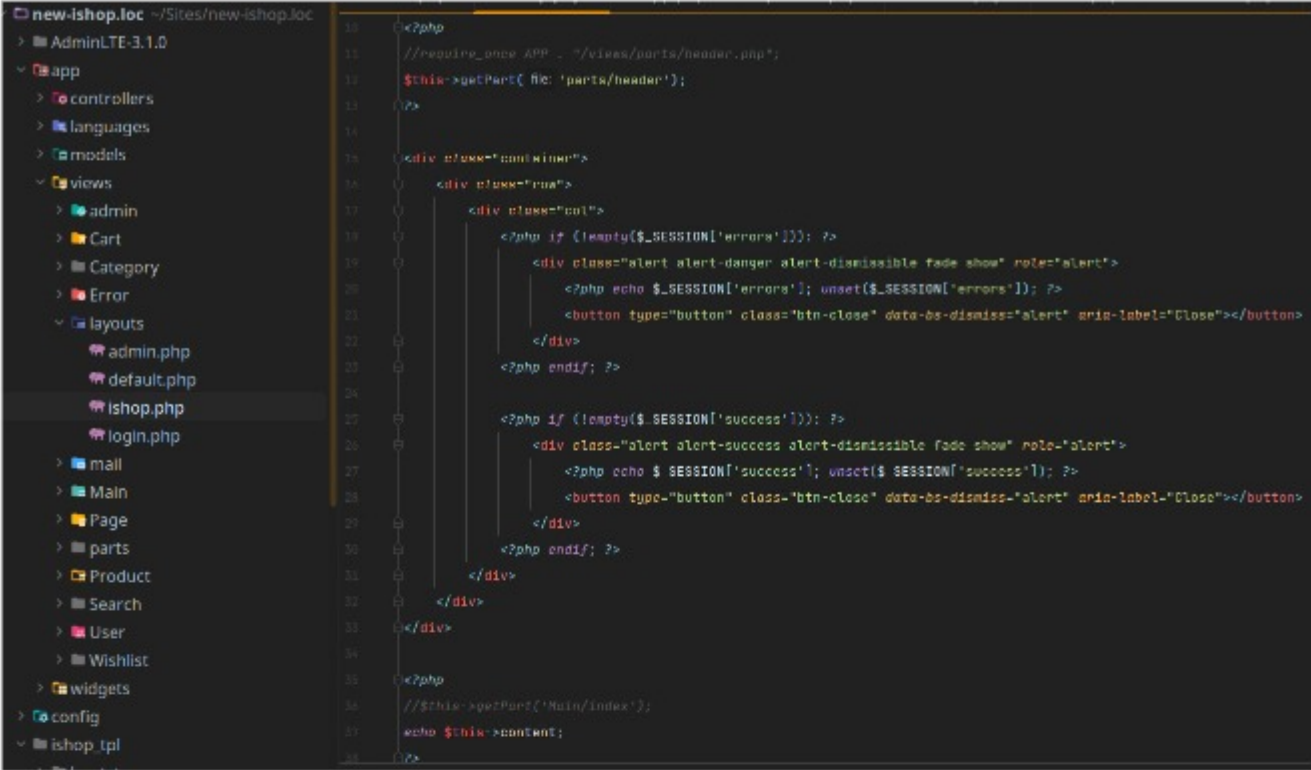
    <div id="product">
        <div class="input-group mb-3">
            <input id="input-quantity" type="text" class="form-control" name="quantity" value="1">
            <button class="btn btn-danger add-to-cart" type="button"
                data-id="<?php echo $product['id'] ?>"><?php echo __('key: product_view_buy'); ?></button>
        </div>
    </div>
</div>
```

```
<?php if (empty($gallery)): ?>
    <?php foreach ($gallery as $item): ?>
        <li class="thumb-additional"><a class="thumbnail" href="<?php echo PATH . $item['img'] ?>" data-effect="mfp-zoom-in">" />
        </li>
    <?php endforeach; ?>
<?php endif; ?>
</ul>

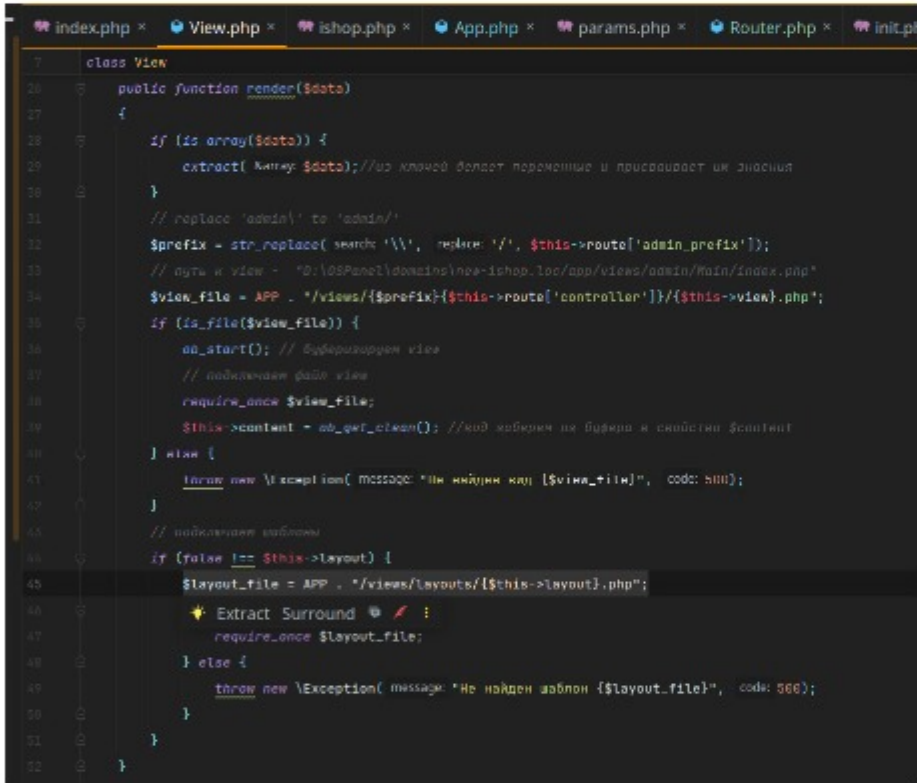
<?php echo $product['content']; ?>
```

7) Вставляем шаблоны на страницу

Получается метод render подключит нужный вид, нужный шаблон и отрисует страницу.
Контент, который будет отрисовывать попадает в \$content.



как раз класс View метод Render подключают нужные страницы:



Мы буферизируем вид:

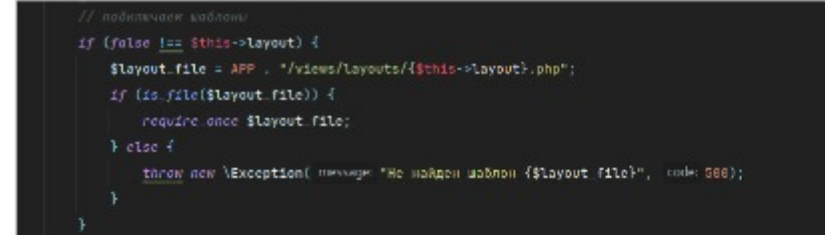
```

// путь к view - "D:\OSPanel\domains\new-ishop.loc/app/views/admin/Main/index.php"
$view_file = APP . "/views/{$prefix}
{$this->route['controller']}/{$this->view}.php";
if (is_file($view_file)) {
    ob_start(); // буферизируем view
    // подключаем файл view
    require_once $view_file;
    $this->content = ob_get_clean(); //вид заберем из буфера в свойство
$content

```

То есть здесь вид **не выводится на экран**, он буферизуется и сохраняется в свойство \$this->content.

Подключаем шаблон:



Шаблон — это общий каркас страницы (html/head/body, шапка/подвал, и т.д.).
Именно внутри layout-файла ты должен вывести <?= \$this->content ?>, чтобы «вклеить» разметку вида.